Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset
Source: UCI Machine Learning Repository
Download Link:
`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Generated Plots with Interpretation

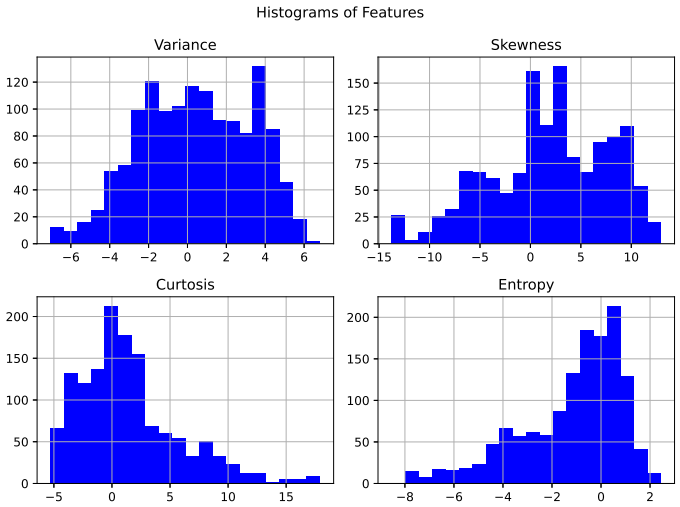

Figure 1: Feature Histograms

Interpretation: The histograms reveal the underlying distribution of each numerical feature. While some features exhibit slight skewness rather than perfect normal distributions, there are no extreme outliers that would severely disrupt the perceptron training process.

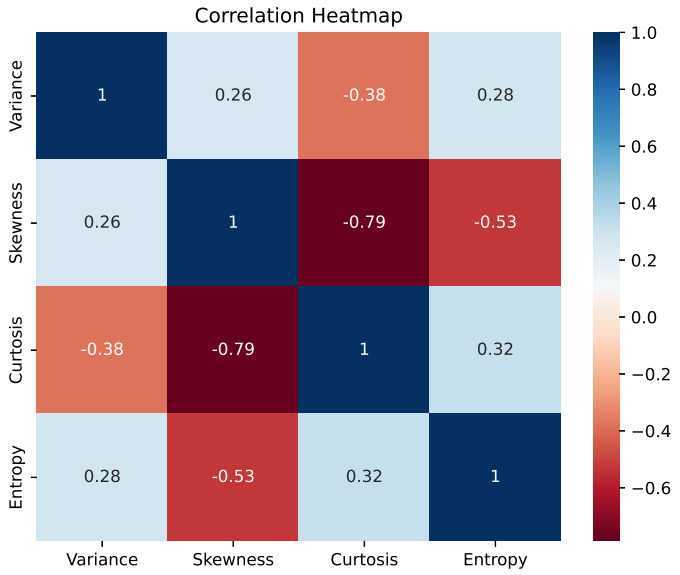

Figure 2: Correlation Heatmap

Interpretation: The correlation heatmap illustrates the linear relationships between pairs of features. No two features are perfectly correlated (e.g., correlation of 1.0 or -1.0), meaning each feature contributes unique, non-redundant information to the model.

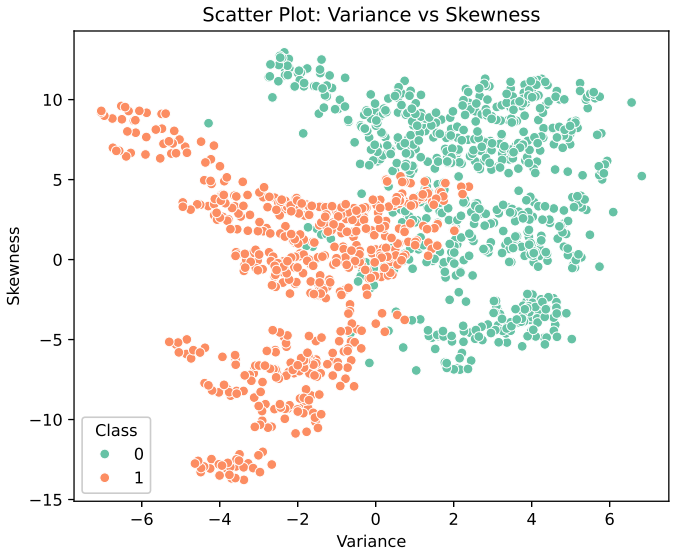

Figure 3: Scatter Plot: Variance vs Skewness

Interpretation: This scatter plot visualizes the distribution of the two classes based on Variance and Skewness. The clusters for authentic and forged banknotes appear largely distinct, indicating that the dataset is highly, if not entirely, linearly separable.

Figure 4: Training Error vs Epoch

Interpretation: The number of misclassified samples decreases sharply in the early epochs as the model updates its weights. The error eventually drops to near zero and stabilizes, successfully verifying the convergence of the perceptron learning algorithm.

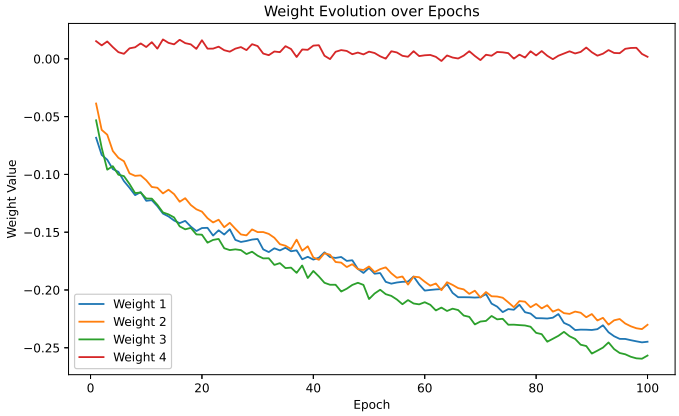

Figure 5: Weight Evolution over Epochs

Interpretation: This plot tracks the individual weight assignments for each feature throughout the training process. After initial fluctuations during the learning phase, the lines become strictly horizontal, demonstrating that the learned weights have permanently stabilized.

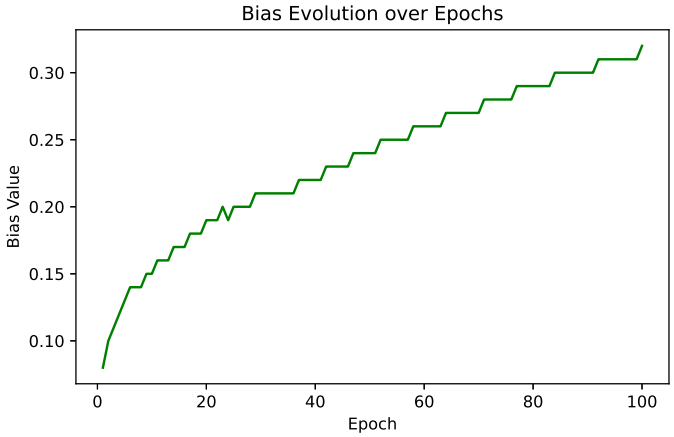

Figure 6: Bias Evolution over Epochs

Interpretation: The bias term shifts during the early epochs to properly shift the decision boundary away from the origin. Just like the weights, it ultimately flattens out, proving that the parameter tuning has reached an optimal state for classification.

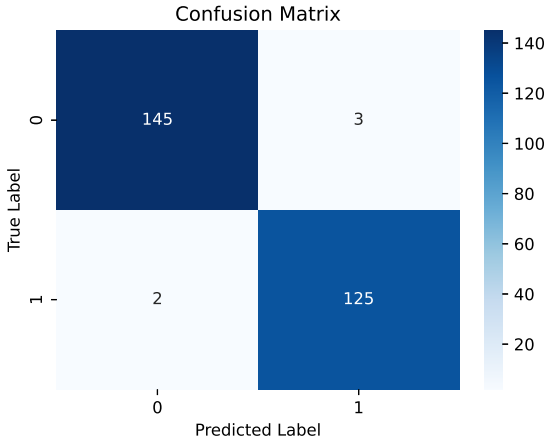

Figure 7: Confusion Matrix

Interpretation: The confusion matrix highlights the classifier's performance on the test set, dominated by high numbers of True Positives and True Negatives on the main diagonal. The off-diagonal cells represent a minimal number of False Positives and False Negatives, confirming high accuracy.

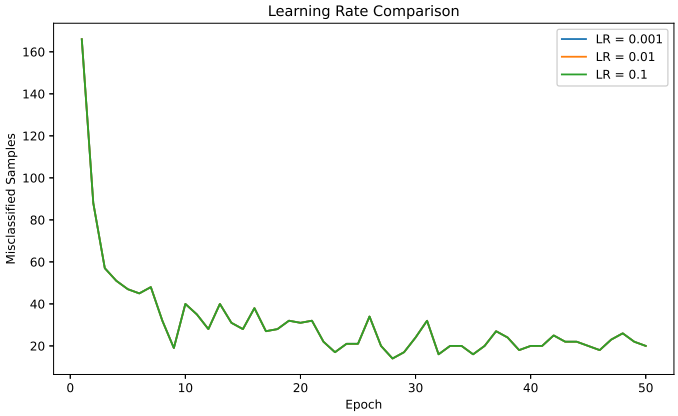

Figure 8: Learning Rate Comparison

Interpretation: This study reveals how different learning rates (η) affect the model's convergence speed. Larger learning rates converge in fewer epochs but risk instability, while the smallest learning rate requires significantly more epochs to reach the same stable error state.

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80% / 20%
Learning Rate	0.01
Epochs	100
Final Weights	[-0.2447762, -0.23011114, -0.25674323, 0.00175867]
Final Bias	0.320
Accuracy	0.9818
Precision	0.9766
Recall	0.9843
F1-score	0.9804

Epoch-wise Learning (Decadal Intervals)

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	166	-0.0683	-0.0387	-0.0532	0.0153	0.0800
10	40	-0.1229	-0.1051	-0.1209	0.0102	0.1500
20	31	-0.1464	-0.1322	-0.1521	0.0161	0.1900
30	24	-0.1559	-0.1499	-0.1703	0.0110	0.2100
40	20	-0.1737	-0.1719	-0.1835	0.0114	0.2200
50	20	-0.1809	-0.1796	-0.2079	0.0062	0.2400
60	22	-0.2005	-0.1928	-0.2106	0.0031	0.2600
70	16	-0.2063	-0.2065	-0.2273	-0.0011	0.2700
80	22	-0.2244	-0.2120	-0.2371	0.0029	0.2900
90	16	-0.2346	-0.2208	-0.2551	0.0057	0.3000
100	23	-0.2448	-0.2301	-0.2567	0.0018	0.3200

9. Discussion

1. Step vs. Sigmoid Activation Functions

The Step activation function maps inputs to binary values (0 or 1), making it simple and good for basic linear classification like our perceptron. But, it is not used in modern deep learning models. This is because its derivative is zero everywhere (and undefined at the origin). Modern neural networks optimize their weights using gradient descent and backpropagation, which needs differentiable functions to calculate errors. Hence Sigmoid Activation Function is used.

2. Custom Implementation vs. Scikit-learn

The custom perceptron achieved an accuracy of 98.18, while Scikit-learn's Perceptron achieved 97.82 . The results are very close, indicating that implementation correctly followed the perceptron learning algorithm., comparison shows that the custom perceptron is mathematically correct, while Scikit-learn provides a reliable and convenient implementation for practical machine learning tasks.

3. Impact of Varying Learning Rates

By repeating the experiment with learning rates of 0.001, 0.01, and 0.1, a clear trade-off emerged. The smallest learning rate (0.001) updated the weights too cautiously, resulting in very slow convergence that struggled to stabilize within the allotted epochs. Conversely, the highest rate (0.1) caused the model to take aggressive steps, leading to erratic oscillations in the training error as it repeatedly overshot the optimal decision boundary. The moderate rate (0.01) served as the "Goldilocks" parameter, yielding a smooth, stable, and timely convergence.

4. The XOR Problem Limitation

A Single Layer Perceptron is fundamentally restricted to solving linearly separable problems—meaning it can only draw a single, straight hyperplane to divide classes. The XOR (Exclusive OR) logic gate generates a non-linear dataset where data points of the same class are positioned diagonally across from one another. Because it is physically impossible to draw a single straight line separating the outputs of an XOR gate, a single-layer network fails. This necessitates the use of hidden layers (Multi-Layer Perceptrons) to distort the input space and resolve non-linear boundaries.

5. The Role of Feature Normalization

Normalizing the Banknote Authentication dataset prior to training was crucial for achieving model convergence. Because raw features often exist on vastly different numerical scales (e.g., Variance might be much larger than Entropy), unscaled data forces the model's loss landscape to become elongated and skewed. Without scaling, the perceptron's weights would heavily bias towards features with larger magnitudes, causing severe instability during training. Normalization centered all features around a mean of zero, creating a symmetric loss landscape that allowed the gradient updates to descend smoothly and efficiently toward the optimal weights.

10. Source Code

Repository Link:

`https://github.com/HarishSNUC1402/DL-Lab-33/tree/main/Lab-1-33`

11. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.